

Beer Review API Documentation

Overview

The Beer Review API allows users to manage and review different types of beers. Users can create new beer entries, retrieve a list of all beers, and access detailed information about each beer. The API is designed to be user-friendly and provides a simple interface for beer enthusiasts.

Features

- **Create Beer:** Users can add new beers to the database with details such as name, style, taste, aroma, color, body, carbonation, alcohol content, and finish.
- **Get All Beers:** Retrieve a list of all beers available in the database, with pagination support.
- **Get Beer by ID:** Access detailed information about a specific beer using its unique identifier.
- **Health Check:** A simple endpoint to check if the API is running smoothly.
- **Metrics:** Access performance metrics for monitoring API usage.

API Endpoints

1. Create a Beer

- **Endpoint:** `POST /api/v1/beers`

Request Body:

```
{
  "name": "Test Beer",
  "style": "IPA",
  "taste": "Fruity",
  "aroma": "Floral",
  "color": "Amber",
  "body": "Medium",
  "carbonation": "Natural",
  "alcohol": 5.5,
  "finish": "Clean"
}
```

- **Response:** Returns the created beer object with a status code of 201.

2. Get All Beers

- **Endpoint:** `GET /api/v1/beers?page=1&pageSize=10`
- **Response:** Returns a list of beers with pagination details.

3. Get Beer by ID

- **Endpoint:** `GET /api/v1/beers/{id}`
- **Response:** Returns the beer object corresponding to the provided ID.

4. Health Check

- **Endpoint:** `GET /health`
- **Response:** Returns a simple "OK" message with a status code of 200.

5. Metrics

- **Endpoint:** `/metrics`
- **Response:** Returns performance metrics for the API.

Getting Started

To get started with the Beer Review API, you can use tools like Postman or cURL to make requests to the endpoints. Ensure you have the necessary permissions and follow the API documentation for proper usage.

Technical Documentation

Technical Overview

The Beer Review API is built using Go and follows a microservices architecture. It utilizes the Gorilla Mux router for handling HTTP requests and Prometheus for monitoring and metrics.

Components

- **Model:** Defines the data structure for beers, including various attributes such as name, style, taste, aroma, etc.
- **Repository:** Interfaces for data storage, including in-memory and PostgreSQL implementations.
- **Usecase:** Contains business logic for managing beers, including creating, retrieving, and paginating beer entries.
- **Delivery:** Handles HTTP requests and responses, including validation and sanitization of input data.
- **Middleware:** Implements request logging and rate limiting.

API Endpoints

1. Create a Beer

- **Method:** POST
- **Path:** /api/v1/beers

Request Body:

json

Copiar código

```
{
  "name": "Test Beer",
  "style": "IPA",
  "taste": "Fruity",
  "aroma": "Floral",
  "color": "Amber",
  "body": "Medium",
  "carbonation": "Natural",
  "alcohol": 5.5,
  "finish": "Clean"
}
```

- **Response:** Returns the created beer object with a status code of 201.

2. Get All Beers

- **Method:** GET
- **Path:** /api/v1/beers?page=1&pageSize=10
- **Response:** Returns a list of beers with pagination details.

3. Get Beer by ID

- **Method:** GET
- **Path:** /api/v1/beers/{id}
- **Response:** Returns the beer object corresponding to the provided ID.

4. Health Check

- **Method:** GET
- **Path:** /health
- **Response:** Returns a simple "OK" message with a status code of 200.

5. Metrics

- **Method:** GET
- **Path:** /metrics
- **Response:** Returns performance metrics for the API.

Conclusion

This documentation provides an overview of the Beer Review API, its features, and how to interact with it. For further details, please refer to the codebase or reach out for support.